MODULE 2

DAY 1 · MODULE 2 OF 8

Discovery at Scale: Understanding an Existing Hierarchical Codebase

Before you write a single spec, you have to understand what's already there.

What You'll Learn

1. Explain why reading code alone doesn't scale on a large, hierarchical codebase
2. Know what to gather before writing a single spec
3. Use MCP to connect an agent to design docs and a bug tracker
4. Place discovery correctly inside Spec Kit and OpenSpec
5. Scope discovery to a service boundary and produce a dependency map

WHY THIS MATTERS

The codebase you're joining isn't the one Module 1's examples assumed.

reading alone stops working exactly here

This module is the missing step before specify: build real understanding first

PART 1

The Challenge

Why reading code alone doesn't scale, and what to gather instead

MODULE 2

THE CHALLENGE

Code Search Finds a File, Not the System Around It

What code search finds
one file, no relationships

What you actually need
50+ interconnected services, hidden dependencies, years of undocumented decisions

the gap between these two is exactly what discovery closes

This gap only widens as a codebase ages, exactly the brownfield case this course uses

MODULE 2

REAL-WORLD CONTEXT

Gather These Four Things Before You Write a Spec

Existing design docs & wikis
architecture, prior decisions, the "why"

Prior specs
what's already been decided and built

Open tracker issues
known, already-reported problems

The code itself
ground truth, but only meaningful alongside the other three

Skipping any one of these is how a spec gets written against a wrong or incomplete picture

CHECKPOINT

Let's Pause and Verify

1 Why doesn't reading code alone scale on a large, hierarchical codebase?

2 Name two things worth gathering before writing a spec, besides the code itself.

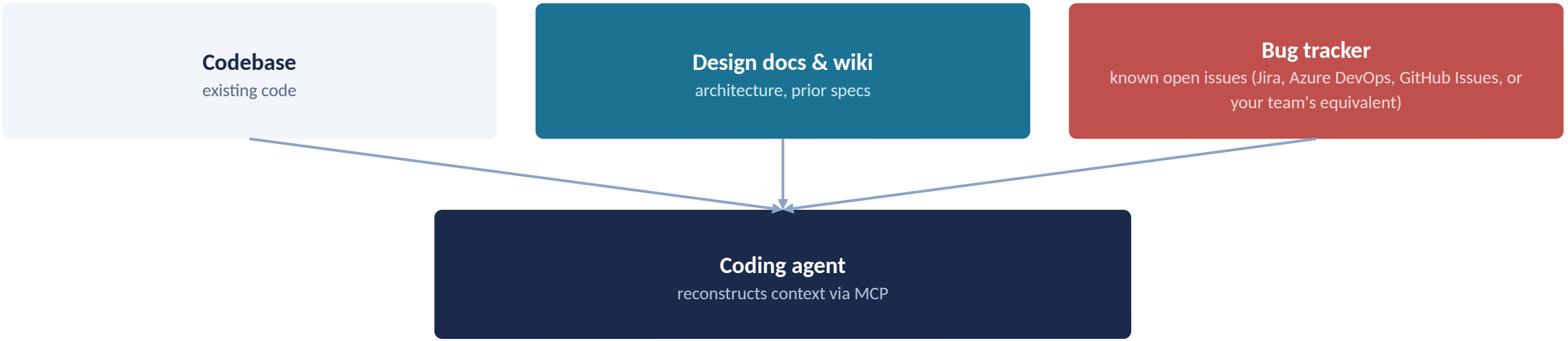
PART 2

Connecting the Agent

MCP as the connective tissue between your agent and everything outside the repo

THE APPROACH

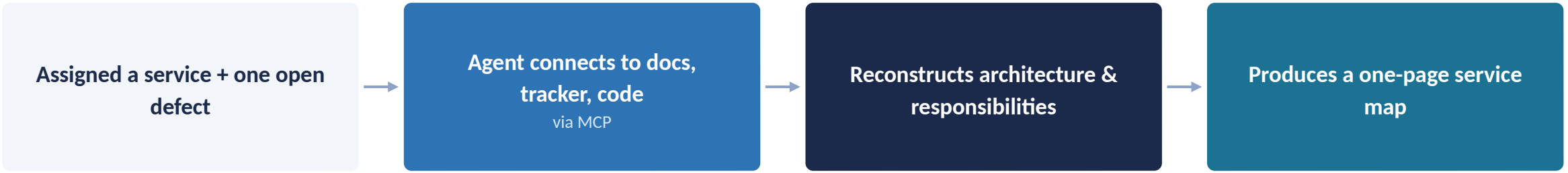
MCP Connects the Agent to What Code Search Alone Would Never Find



MCP is the connective tissue between your agent and everything outside the repo itself

REAL-WORLD CONTEXT

This Is What Discovery Actually Looks Like End to End



Four steps, and the output of the last one is what you carry into Part 3

MODULE 2

WHERE THIS FITS

Spec Kit Assumes Discovery Already Happened, OpenSpec Has a Command for It

discovery happens upstream of Spec Kit's CLI, or natively inside OpenSpec's

Either way, discovery's output is the context you hand to next module's spec

CHECKPOINT

Let's Pause and Verify

1 What does MCP actually connect the agent to, in this module's context?

2 Which framework has a dedicated discovery command, and what is it called?

PART 3

Scoping the Work

One service boundary at a time, and the map that carries forward

MODULE 2

SCOPE IT

Scope Discovery to One Service Boundary, Not the Whole Repository

scoping is what makes discovery finishable

A finished map for one service beats a half-finished map for the whole system

MODULE 2

THE ARTIFACT

The Service Map Is a Precursor Artifact, Not the Destination

What's in this service
responsibilities, entry points

What it depends on
upstream services & shared modules

What depends on it
downstream consumers, blast radius

The defect you pulled in
carries forward into Module 3's spec

One page, four things, that's the entire deliverable for this module

MODULE 2

COMMON PITFALLS

The instinct to be thorough is what makes discovery stall at this scale.

the fix isn't working harder, it's scoping tighter

If a discovery pass has been running for days with no map yet, the scope was too big, not the effort too small

CHECKPOINT

Let's Pause and Verify

1 Why scope discovery to one service boundary instead of the whole repo?

2 Name one thing a service/dependency map should capture besides its own responsibilities.

Key Takeaways

1. Discovery precedes specification, always, on a brownfield codebase: you can't write requirements against a system you don't yet understand.
2. Gather design docs, prior specs, tracker issues, and code together, no single source is sufficient on its own.
3. MCP is the connective tissue between your agent and everything outside the repo: docs, trackers, and prior decisions.
4. OpenSpec's explore command is a natural fit for this step; Spec Kit expects discovery to happen upstream of it, in your agent rather than in the CLI.
5. Next: Module 3, Establishing Governance and Writing a Clarified Specification, starts from the service map and defect you produce today.